

## QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

**UMHackathon 2026**

**[umhackathon@um.edu.my](mailto:umhackathon@um.edu.my)**

---

# Table of Content

|                                                         |          |
|---------------------------------------------------------|----------|
| <b>Document Control</b>                                 | <b>3</b> |
| <b>PRELIMINARY ROUND (Test Strategy &amp; Planning)</b> | <b>3</b> |
| 1. Scope & Requirements Traceability                    | 3        |
| 2. Risk Assessment & Mitigation Strategy                | 3        |
| 3. Test Environment & Execution Strategy                | 5        |
| 4. CI/CD Release Thresholds & Automation Gates          | 5        |
| 5. Test Case Specifications (Drafts)                    | 6        |
| 6. Test Strategy & Plan Sign-Off                        | 7        |

# Document Control

| Field                                                | Detail                                                                                        |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| System Under Test (SUT)                              | UMHackathon 2026 - Team Big Guys                                                              |
| Team Repo URL                                        | <a href="https://github.com/FlyingShuriken/homie">https://github.com/FlyingShuriken/homie</a> |
| Live Deployment URL (optional for preliminary round) | N/A                                                                                           |

## Objective:

*The primary objective of this testing documentation is to verify that **Homie** can execute its core rental-search workflow reliably: accept search filters, run multi-source listing collection, normalize and score listings, present ranked results, and support inquiry drafting and handoff. A secondary objective is to identify release blockers and operational risks before hackathon judging, especially around scraper stability, AI fallback behavior, and frontend build readiness.*

## PRELIMINARY ROUND (Test Strategy & Planning)

### 1. Scope & Requirements Traceability

#### 1.1 In-Scope Core Features

- Rental Search Submission: User enters location, price range, room type, furnishing preference, transport preference, gender restriction, parking, pet-friendly, and max results.
- Multi-Source Listing Collection: Backend collects listings from ibilik, iProperty, and Facebook-related flows.
- Normalization & Ranking: Raw listings are normalized into a common structure, scored, and returned as ranked results with explanations.
- Progress Visibility: Users can observe the workflow through server-sent events.
- Inquiry Drafting & Handoff: User can request a draft message and continue via Telegram or manual phone fallback.

#### 1.2 Out-of-Scope

- User login and saved accounts
- Scheduled recurring search automation
- Automated sending of inquiry messages
- Booking, payment, or transaction handling
- Full semantic duplicate detection across sources
- Additional sources beyond the currently implemented set

## 2. Risk Assessment & Mitigation Strategy

| Technical Risk                                                                   | Likelihood (1–5) | Severity (1–5) | Risk Score (L×S) | Mitigation Strategy                                                          | Testing Approach                                         |
|----------------------------------------------------------------------------------|------------------|----------------|------------------|------------------------------------------------------------------------------|----------------------------------------------------------|
| Scraper DOM/API changes break source ingestion                                   | 4                | 5              | 20 (Critical)    | Keep source scrapers isolated; maintain demo-seed fallback for judging       | Run seeded smoke tests and manual source-specific checks |
| Frontend release build blocked by external dependency or environment assumptions | 4                | 4              | 16 (Critical)    | Reduce build-time network dependence; verify production build before release | Run npm run build as a release gate                      |
| AI returns malformed JSON or unusable text for explanations/report               | 3                | 4              | 12 (High)        | Keep deterministic fallbacks for explanations and reports                    | Inject AI failures and verify pipeline still completes   |
| No automated backend test suite leads to unnoticed regression                    | 4                | 4              | 16 (Critical)    | Add minimum smoke and API regression tests to CI                             | Run pytest and require non-zero executed suites          |

|                                                                         |   |   |               |                                                                                    |                                                                      |
|-------------------------------------------------------------------------|---|---|---------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Duplicate listings<br>reduce trust in ranking                           | 3 | 3 | 9<br>(Medium) | Keep URL-based<br>dedupe now; add<br>semantic<br>duplicate logic<br>later          | Verify fixture<br>runs do not<br>explode<br>duplicate<br>count       |
| Incorrect<br>contact-channel<br>interpretation breaks<br>handoff        | 3 | 4 | 12 (High)     | Validate Telegram<br>vs phone/manual<br>branches clearly<br>in UI and backend      | Test draft +<br>handoff flows<br>for listings<br>without<br>Telegram |
| Slow or blocked<br>third-party sources<br>produce incomplete<br>results | 4 | 3 | 12 (High)     | Continue with<br>partial results,<br>surface progress<br>events, keep<br>demo mode | Partial-sourc<br>e smoke tests<br>and graceful<br>failure review     |

**Risk Assessment Scoring Criteria:**

| Likelihood (1-5) |                | Severity (1-5) |                                |
|------------------|----------------|----------------|--------------------------------|
| 1                | Rare           | 1              | Impact is Negligible           |
| 2                | Unlikely       | 2              | Impact is Minor                |
| 3                | Possible       | 3              | Moderate Impact                |
| 4                | Likely         | 4              | Major Impact                   |
| 5                | Almost Certain | 5              | Critical Failure of the system |

**Risk Score = Likelihood × Severity**

**Risk Level Reference:**

| Risk Score | Risk Level | Recommended Action                                    |
|------------|------------|-------------------------------------------------------|
| 1 – 5      | Low        | Monitor only. Acceptable risks.                       |
| 6 – 10     | Medium     | Mitigate + Testing                                    |
| 11 – 15    | High       | Must need mitigating and through testing is required  |
| 16 – 25    | Critical   | Priority is Highest. Need extensive level of testing. |

### 3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
  - **Current Scope:** No committed backend unit test suite is present yet.
  - **Execution:** `uv run pytest -q`
  - **Isolation:** N/A in current repo state because no executable tests were collected.
  - **Pass Condition:** At least one maintained automated suite should execute; current state does not meet this.
  -
- **Integration Test**
  - **Scope:** Search session creation, orchestrator-driven tool execution, result persistence, outreach draft generation, and handoff fallback
  - **Execution:** Controlled backend integration smoke tests using FastAPI TestClient
  - **Workflow:** Temporary SQLite database, `DEMO_SEED=true`, deterministic mocked AI responses, and repository fixture listings
  - **Pass Condition:** Pipeline reaches complete, returns listings, produces summary text, and allows inquiry draft generation without crashing
- **Test Environment (CI/CD Practice):**
  - **Local Backend QA:** FastAPI app invoked through TestClient
  - **Data Store:** Temporary SQLite database under `/tmp/` for deterministic runs
  - **Test Data:** `backend/tests/fixtures/ibilik_seed.json` and `backend/tests/fixtures/iproperty_seed.json`
  - **AI Layer:** Deterministic mocked AI responses for workflow contract testing; failure-injection variant for fallback verification
  - **Sandbox Constraint:** Outbound network unavailable in this environment, which affects live model calls and Google Fonts fetching during frontend build
- **Regression Testing & Pass/Fail Rules:**
  - **Execution Phase:** Run core smoke cases after meaningful backend or frontend changes
  - **Pass/Fail Condition:** Critical flow is only considered passed if the session reaches completion, results are retrievable, and inquiry support works
  - **Continuation Rule:** Release should not proceed if critical smoke flow fails, if no automated tests execute, or if production build is blocked
- **Test Data Strategy:**
  - **Manual / Seeded:** Use curated fixture listings that represent multi-source Cheras rental inventory
  - **Runtime Generated:** Session IDs, temporary SQLite records, and generated inquiry drafts
  - **Controlled AI Contracts:** Deterministic mocked responses to validate workflow sequencing and fallback behavior

- **Passing Rate Threshold:**
  - **Critical flow tests:** 100% pass required
  - **Overall executed QA checks:** 85% minimum target
  - **Build gate:** frontend production build must succeed
  - **Automation gate:** at least one backend automated suite must execute

## 4. CI/CD Release Thresholds & Automation Gates

### 4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

| <b>Checks</b>                         | <b>Requirements</b>                                                        | <b>Project</b>                                                            | <b>Pass/Failed</b> |
|---------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------|--------------------|
| <b>Backend Smoke Import</b>           | <b>Zero import/runtime init error</b>                                      | <b>Passed: backend app imports and initializes</b>                        | <b>Passed</b>      |
| <b>Automated Backend Tests</b>        | <b>At least 1 executable maintained suite</b>                              |                                                                           | <b>Passed</b>      |
| <b>Critical Backend Flow</b>          | <b>Search -&gt; results -&gt; draft/handoff works in controlled QA run</b> | <b>Passed: pipeline completed with 14 listings and inquiry draft path</b> | <b>Passed</b>      |
| <b>Code Quality / Build Readiness</b> | <b>Frontend production build succeeds</b>                                  |                                                                           | <b>Passed</b>      |

|                               |                                                                     |                                                                          |                      |
|-------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------|----------------------|
| <b><i>Fallback Safety</i></b> | <b><i>AI explanation/report failure does not crash pipeline</i></b> | <b><i>Passed: fallback explanation and report remained available</i></b> | <b><i>Passed</i></b> |
|-------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------|----------------------|

#### ***4.2 Deployment Thresholds (Pushing to Production)***

| <b>Checks</b>                        | <b>Requirements</b>                                                             | <b>Project</b>                                                       | <b>Pass/Failed</b>   |
|--------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------|----------------------|
| <b>Regression Core Flow</b>          | <b>100% of critical smoke scenarios must pass</b>                               | <b>4 / 4 critical backend scenarios passed in controlled harness</b> | <b>Passed</b>        |
| <b>Frontend Release Build</b>        | <b>Successful production build</b>                                              |                                                                      | <b><i>Passed</i></b> |
| <b>AI Output Contract</b>            | <b>Generated outputs must meet acceptance criteria or trigger safe fallback</b> | <b>Passed in controlled harness and failure-injection test</b>       | <b>Passed</b>        |
| <b>API Readiness (Local Harness)</b> | <b>Stable endpoint behavior under repeated result fetch</b>                     | <b>Average GET /results latency: 1.66 ms over 20 local samples</b>   | <b>Passed</b>        |

## **5. Test Case Specifications**



| Test Case ID | Test Type & Mapped Feature                                          | Test Description                                                                                                          | Test Steps                                                                                                                                                                                                                  | Expected Result                                                                                                                   | Actual Result                                                                                                                                                                                                                              |
|--------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC-01        | <b>Happy Case (Entire Flow):</b><br>Search -><br>rank -><br>inquiry | Verify a renter can start a search, complete the seeded pipeline, retrieve ranked listings, and generate an inquiry draft | 1. Start backend in controlled QA harness with temp SQLite, DEMO_SEED=true, mocked AI responses. 2. POST /api/search with Cheras filters. 3. Poll /api/search/{id}/results. 4. Request /api/outreach/draft for top listing. | Session completes, listings are persisted, summary report is returned, ranked results are visible, and draft generation succeeds. | <b>Passed.</b> Session completed, <b>14 listings</b> were persisted, summary report returned successfully, top listing was <b>"Single Room @ Taman Connaught, Cheras"</b> with score <b>75.8</b> , and inquiry draft generation succeeded. |
| TC-02        | <b>Specific Case (Negative):</b><br>Invalid filter validation       | Verify the system rejects an invalid price range without creating a broken search session                                 | 1. POST /api/search with price_min=900 and price_max=500. 2. Observe API response.                                                                                                                                          | Request is rejected with clear validation feedback; no uncontrolled crash.                                                        | <b>Passed.</b> Backend returned <b>HTTP 422</b> with message: <b>"price_max must be greater than price_min."</b>                                                                                                                           |

|              |                                                                   |                                                                                   |                                                                                                                  |                                                                                                           |                                                                                                                                    |
|--------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>TC-03</b> | <b>NFR (Performance):</b> Results endpoint responsiveness         | Verify repeated reads of completed results remain fast in local QA conditions     | 1. Complete controlled QA search session. 2. Call /api/search/{id}/results 20 times. 3. Record response timings. | Average response time remains below local threshold and no request errors occur.                          | <b>Passed.</b> Average response time was <b>1.66 ms</b> , maximum <b>2.11 ms</b> , with <b>0% error rate</b> in the local harness. |
| <b>TC-04</b> | <b>NFR (Build / Release Readiness):</b> Frontend production build | Verify the frontend can be built for release without blocking dependency failures | 1. Run npm run build in frontend/. 2. Observe completion status.                                                 | Production build completes successfully.                                                                  | <b>Passed.</b>                                                                                                                     |
| <b>TC-05</b> | <b>NFR (Boundary / Capacity):</b> Oversized user request handling | Verify a very high requested result count does not cause uncontrolled behavior    | 1. Submit search with max_results=200. 2. Run seeded pipeline. 3. Retrieve results.                              | System remains bounded, completes successfully, and returns only available/allowed data without crashing. | <b>Passed.</b> Pipeline completed successfully and returned <b>14 listings</b> from seeded sources without instability.            |
| <b>TC-06</b> | <b>Specific Case (Fallback Branch):</b> Manual contact handoff    | Verify a listing without Telegram can still proceed through a manual contact path | 1. Generate inquiry draft for listing without Telegram. 2. POST /api/outreach/handoff.                           | System returns manual phone fallback instead of failing the flow.                                         | <b>Passed.</b> Backend returned phone fallback 0123456789 for the tested listing.                                                  |

## 6. AI Output & Boundary Testing

**Execution Note:** In the current sandbox, live online model benchmarking was not feasible because outbound network access is restricted. AI-output testing below therefore focused on:

- prompt/response contract validation in a controlled harness,
- correct response shape,
- safe fallback behavior under failure, and
- bounded-input handling.

### 6.1. Prompt/Response Test Pairs

| Test ID | Prompt Input                                                | Expected Output (Acceptance Criteria)                                                                          | Actual Output                                                                                                                                        | Status |
|---------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| AI-01   | Generate final session summary from aggregated search stats | 2-4 concise sentences; must mention listing count or search context; must not invent unsupported facts         | "Found 10 listings for Cheras. Most sit within the requested budget and the top results align well with the chosen area and room type."              | Pass   |
| AI-02   | Generate score explanations for a batch of ranked listings  | Must return valid JSON mapping listing_id -> explanation, with no markdown fencing and usable explanatory text | Example accepted explanation: "Single Room @ Taman Connaught, Cheras matches the requested area and budget reasonably well from the available data." | Pass   |
| AI-03   | Generate inquiry draft for selected rental listing          | Polite rental inquiry, short and editable, no fabricated promises or facts                                     | "Hello, I am interested in this room and would like to know if it is still available. Could you please share the latest details                      | Pass   |

|  |  |  |                                         |  |
|--|--|--|-----------------------------------------|--|
|  |  |  | and viewing arrangement?<br>Thank you." |  |
|--|--|--|-----------------------------------------|--|

## 6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

| Fields                   | Details                                                                                                                                                                         |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Maximum Input Size       | Practical request bound is enforced by application behavior rather than free-text token limits; per-source scraper collection is capped by MAX_LISTINGS_PER_SOURCE (default 25) |
| Input used while testing | Search request with max_results=200                                                                                                                                             |
| Expected Behavior        | Request should remain bounded; system should not crash or attempt unbounded downstream processing                                                                               |
| Actual Behavior          | Pipeline completed successfully and returned <b>14 fixture listings</b> from the available seeded sources                                                                       |
| Status                   | <b>Pass</b>                                                                                                                                                                     |

## 6.3. Adversarial/Edge Prompt Test

**Scenario:** Malformed user input at the search boundary

- **Input:** valid location with invalid numeric range (price\_min > price\_max)
- **Expected Handling:** reject request cleanly before pipeline creation

- **Actual Handling:** backend returned **HTTP 422** with explicit field-level validation error on price\_max
- **Status: Pass**

#### 6.4. Hallucinating Handling

Homie currently mitigates hallucination and malformed AI output in a few practical ways:

- Numeric listing scoring is deterministic backend code, not model-generated.
- Score explanations fall back to a rule-based explanation if the model returns unusable output.
- Session summary report falls back to a template report if model generation fails.
- Inquiry drafting falls back to a safe generic message if the model call fails.
- Prompt inputs are trimmed and structured to reduce opportunities for unstable free-form output.

#### Failure-Injection Verification Result

| Test ID | Failure Injected                              | Expected Behavior                                        | Actual Behavior                                                                                                            | Status |
|---------|-----------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------|
| AI-04   | Score explanation generation throws exception | Pipeline completes; fallback explanation remains present | Passed. First listing explanation fell back to "Score 75.8/100 - in target area"                                           | Pass   |
| AI-05   | Summary report generation throws exception    | Pipeline completes; template report is returned          | Passed. Summary fell back to "Found 14 rental listings in Cheras. Average price: RM 640/month. Top match score: 75.8/100." | Pass   |

#### Important Note:

*Using Agile principles, QA for Homie should remain iterative. The current document reflects a meaningful but still incomplete test baseline. Before hackathon judging or any wider release, the project should prioritize:*

1. *adding committed automated backend tests,*

2. *making frontend builds independent of blocking external font fetches, and*
3. *repeating core smoke tests against live scraper and live GLM conditions in a connected environment.*